

ATNLP Exam Guide Speed Notes

Chinese explanations with English exam terms

Generated for revision

2026-05-20

Contents

使用方式	2
1. Machine Translation	3
2. Question Answering	5
3. Data and Synthetic Data	7
4. Evaluation	9
5. Long Context Modelling	11
6. RAG	13
7. Multilingual LLMs	15
8. Bias and Ethics	17
9. Safety, Watermarking and Adversarial Attacks	19
10. Instruction Finetuning, RLHF and Basic RL	21
11. Summarisation	23
12. Parameter-Efficient Finetuning	25
13. Distillation	27
14. Reasoning During Decoding	29
15. Mechanistic Interpretability	31
16. Syntactic Parsing	33
17. Perceptually-Grounded NLP	35

18. Vision-Language Models	37
19. Text-Only AI Agents	39
20. Vision-Based Agents and OpenCUA	41
Final rapid recap	43

使用方式

这份讲义按 exam guide 的 topic 顺序整理。每个 topic 都压成五块：Core thesis、Must-know points、Exam contrasts/formulas/workflows、Pitfalls、30-second exam template。因为考试是英文作答，关键 exam terms 都尽量用 English noun phrases；中文只负责解释直觉和关系。考前优先背每节的 Core thesis 和 30-second exam template，再用 contrasts/formulas/workflows 补细节。

1. Machine Translation

Core thesis: Machine Translation 的核心是在 lexical ambiguity、morphological complexity、word order differences 和 data imbalance 的限制下，用 parallel corpora 训练 encoder-decoder 或 LLM systems，并用 human evaluation 与 automatic evaluation 判断 translation 是否 adequate and fluent。

Must-know points

- MT challenge 1 是 lexical ambiguity，例如 bank 需要 context disambiguation 才知道是银行还是河岸。
- MT challenge 2 是 morphological complexity，morphologically rich languages 中一个词可能包含 root、number、case、tense 等多层信息。
- MT challenge 3 是 word order differences，不同 languages 有 SVO、VSO、postpositions、adjective position 等差异。
- Parallel corpora 是 traditional MT 和 NMT 的核心 training data，来源包括 Bible、Hansard、Europarl、UN、web-crawled corpora。
- Web parallel data extraction pipeline: language identification、cleaning、document alignment、sentence splitting、sentence alignment、deduplication、filtering。
- Low-resource languages 常遇到 few web pages、poor language identification、low text quality、noisy sentence alignment。
- NMT 学习 $p(T \mid S)$ ，output 同时依赖完整 source sentence 和 previous target tokens。
- 标准 MT architecture 是 encoder-decoder Transformer: encoder 编码 source sentence，decoder 通过 cross-attention 访问 source representations，并 autoregressively generate target sentence。
- Human evaluation 看 adequacy 和 fluency; automatic evaluation 用于 low-cost system comparison。
- LLM translation 进入 pretrain-prompt paradigm，但 low-resource translation、domain transfer、evaluation reliability 仍不是 solved problem。

Exam contrasts/formulas/workflows

- Translation objective: $T^* = \arg \max_T p(T \mid S)$ 。
- Sequence factorisation: $p(T \mid S) = \prod_i p(y_i \mid y_{<i}, x_1, \dots, x_m)$ 。
- Phrase-based MT vs NMT: 前者依赖 hand-crafted features、sparse log-linear model 和复杂 search; 后者端到端、continuous representations、conditioned on full source sentence。
- Adequacy: 是否保留 source sentence meaning; Fluency: target language 是否 natural、grammatical、idiomatic。
- ChrF: 字符 n-gram precision/recall 的 F-score，常比只看词级 overlap 的 BLEU 更适合 morphologically rich languages。
- COMET: 用 cross-lingual pretrained model，并在 WMT human evaluation data 上 fine-tune，更贴近 human judgement，但仍有 generalisation limitations。

Pitfalls

- Automatic metric 高不等于 translation 一定好; automatic metrics 可能漏掉 semantic errors、omissions 或 additions。
- 不要只说 MT data 来自 web; 考试要写 data extraction、alignment、deduplication、filtering pipeline。
- Decoder 不是只看 target history, 它还通过 cross-attention 看 source sentence。
- BLEU/ChrF/COMET 不是同一类: BLEU/ChrF 偏 overlap, COMET 是 learned metric。

30-second exam template

Machine Translation 是给定 source sentence S , 寻找最可能 target sentence T^* 的任务, 即最大化 $p(T \mid S)$ 。它困难在 lexical ambiguity、morphological complexity、word order differences 和 low-resource data scarcity。现代 NMT 通常用 encoder-decoder Transformer: encoder 表示 source sentence, decoder 在 target history 和 cross-attention 条件下 generate token by token。Training 主要依赖 parallel corpora, web data 需经过 language identification、cleaning、document/sentence alignment、deduplication 和 filtering。Evaluation 上, human adequacy/fluency judgement 仍是 gold standard; ChrF、BLEU、COMET 等 automatic metrics 适合 large-scale comparison, 但不能完全替代 human evaluation。

2. Question Answering

Core thesis: Question Answering 从 “给定 context 找 answer” 的 reading comprehension, 发展到 “先 retrieve evidence, 再 extract/generate answer” 的 open-domain QA, 核心始终是 question、evidence 和 answer 之间的 alignment。

Must-know points

- QA 是自动回答 natural-language questions 的系统, 是测试 language understanding 的重要 test bed。
- Reading Comprehension 形式是 $P, Q \rightarrow A$: 给定 passage 和 question, 输出 answer。
- Extractive QA 要求 answer 是 context 中连续 span, 常预测 start/end indices。
- SQuAD 来自 Wikipedia passages 和 crowdsourced questions, answer 是 passage 中短 span。
- SQuAD 常用 Exact Match 和 token-level F1 evaluation。
- SQuAD limitations 是 clean setup、short passages、mostly span answers, 缺少 real retrieval、ambiguity 和 unanswerable questions。
- Natural Questions 使用真实 Google queries 和完整 Wikipedia 页面, 更接近真实搜索 QA。
- Open-domain QA 不给定 passage, 需先从 large document collection retrieve evidence。
- Retriever-reader pipeline: retriever 找 top-k passages, reader/generator extract answer 或 generate answer。
- Fusion-in-Decoder 将多个 retrieved passages 分别编码, 在 decoder 中 fuse evidence and generate answer。

Exam contrasts/workflows

- RC vs ODQA: RC 给定 passage; ODQA 不给 passage, 要先 perform retrieval。
- SQuAD vs Natural Questions: SQuAD 更干净、short passage、span answer; NQ 更真实、full page、long/short answer、no-answer 和 ambiguity。
- Sparse retrieval vs dense retrieval: 前者 BM25/TF-IDF 词面匹配; 后者 dense vectors 和 dot product 做 semantic matching。
- Extractive QA 输出约束: $1 \leq start \leq end \leq n$ 。
- FiD workflow: retrieve top-k passages $\rightarrow$ 每个 question + passage 独立 encoder $\rightarrow$ decoder attends over all representations $\rightarrow$ generate answer。

Pitfalls

- SQuAD 上超过 human performance 不等于 QA 被解决, 因为 real QA 还有 retrieval、多 evidence 和 open-ended generation 问题。
- EM 很严格; 语义正确但表达不同也可能 0 分。F1 给 token overlap 的部分分。

- ODQA 的错误可能来自 retriever, 也可能来自 reader/generator。
- FiD 的 “fusion” 发生在 decoder, 不是在 encoder 里把所有 passages 粗暴拼接成一段。

30-second exam template

Question Answering 要求系统 understand question、locate evidence 并 output answer。Reading Comprehension 中 input 是 passage P 和 question Q , output 是 answer A ; extractive QA 通常预测 answer span 的 start/end。SQuAD 推动了 neural RC, 但它缺少 real retrieval 和 unanswerable questions。Natural Questions 更真实, 因为 questions 来自 Google queries, documents 是完整 Wikipedia pages。Open-domain QA 更难, 因为系统必须先 retrieve top-k evidence, 再由 reader/generator answer。Fusion-in-Decoder 是 generative ODQA 方法: 多个 retrieved passages 分别编码, decoder 在 generation 时 fuse all evidence。

3. Data and Synthetic Data

Core thesis: LLM capability 很大程度由 data 决定，关键不是“越多越好”，而是 data sources、data selection、deduplication、data mixing、synthetic data 与 data ethics 共同决定 training signal quality。

Must-know points

- Data stages: pre-training data 学 general language/knowledge; mid-training/annealing data 强化 high-quality capabilities; post-training data 学 instruction following、preference 和 safety。
- Main sources: Wikipedia 质量高但 representativeness 有限; Common Crawl 规模大但 noise、duplicates、copyright 和 geographic bias 严重; code data 要看 license。
- Data selection goals: 提升 performance、降低 training cost、保护 evaluation integrity、减少 toxicity/bias/PII risks。
- Common filtering pipeline: language filtering -> heuristic filtering -> quality/classifier filtering -> domain filtering -> deduplication -> toxic/PII filtering -> data mixing。
- Deduplication: exact dedup 去 exact duplicates; approximate dedup 去 near duplicates; MinHash 用 Jaccard signature 高效找重复。
- FineWeb 从 Common Crawl 构建 high-quality web corpus，核心思想是 quality trumps quantity。
- FineWeb-Edu 用 strong LLM generate educational-value synthetic labels，再训练 lightweight classifier 筛出 educational content。
- Synthetic instruction data 是由 models、programs 或 automatic pipelines 生成的 instruction-response pairs，用于 instruction tuning 或 distillation。
- Synthetic data methods 包括 stronger-model distillation、self-generation + verification、Evol-Instruct、procedural generation。
- Data ethics 包括 copyright、PII、toxicity、dialect over-filtering、group under-representation 和 dominant-culture over-representation。

Exam contrasts/workflows

- Pre-training vs post-training: 前者学语言/知识分布，后者学“如何按人类意图回答”。
- Heuristic filtering vs classifier filtering: 前者 cheap/fast 但 shallow; 后者更能捕捉 quality 但依赖 training labels。
- Exact vs approximate dedup: 完全相同 vs 模板/轻微 paraphrasing 近似相同。
- MinHash 流程: 文档转 token/n-gram 集合 -> 多个 hash 得签名 -> 签名匹配比例近似 Jaccard -> 删除 near duplicates 簇。
- Synthetic data 优势是低 cost 扩展任务 coverage; 风险是继承 teacher 错误、mode collapse、放大 bias。

Pitfalls

- 不要把 “more data” 当成必然更好；low-quality duplicate data 会浪费 compute 并伤害 generalisation。
- Toxicity filtering 不是删得越多越好，可能误伤 marginalised dialects 和讨论 discrimination 的文本。
- PII handling 要区分 public knowledge 和 private data。
- FineWeb-Edu 不是 pure synthetic corpus，而是用 synthetic labels 做 real-data selection。

30-second exam template

Data 是 LLM 的核心 training signal，决定 knowledge、language patterns、bias 和 safety behaviour。Training data 来自 Wikipedia、Web/Common Crawl、code 等，但必须经过 language filtering、heuristics、quality filtering、deduplication、toxicity/PII filtering 和 data mixing。FineWeb 展示了 high-quality filtering 和 deduplication 比盲目扩大 scale 更重要；FineWeb-Edu 用 LLM-generated educational-value labels 来筛选 high-quality educational data。Synthetic instruction data 可 low-cost 扩展 instruction tuning，但需要 verification、diversity control 和 human data mixing，同时注意 copyright、privacy 和 representational ethics。

4. Evaluation

Core thesis: LLM evaluation 没有 single true metric, 必须按 use case、answer type、ground truth、metric、prompt 和 deployment risk 做 multi-dimensional evaluation。

Must-know points

- Evaluation goals: compare systems、判断 change 是否有效、判断 evaluation 是否适合 deployment。
- Evaluation framework dimensionality: answer type、ground truth、metric、prompt formulation、deployment criteria。
- Intrinsic evaluation 直接评 intrinsic tasks, 如 perplexity、cloze; 便宜但不能代表 factuality、reasoning 和 alignment。
- Task-based benchmarks 如 MMLU、MMLU-Pro、GPQA、HLE 用于标准化比较。
- Benchmark saturation: 模型接近满分后, benchmark 失去区分能力。
- Contamination: test set 进入 training data, 导致模型靠 memorisation 得高分, 破坏 generalisation evaluation。
- LM Arena: 真实 users 比较两个匿名模型回答, 用 pairwise preference 聚合 open-ended response quality。
- Elo: 表现高于预期就涨分, 低于预期就降分, 适合 pairwise comparison。
- LLM-as-a-Judge: 用 strong model evaluate outputs, 可扩展、有 explanations, 但有 position bias、self-preference、prompt sensitivity。
- Real deployment evaluation 要看 individual-instance errors、failure modes、risk severity, 不只看 average score。

Exam contrasts/formulas

- Intrinsic vs task-based: 前者测语言建模等内部能力; 后者测具体任务表现。
- Closed-ended vs open-ended: 有 gold answer 可用 accuracy/F1; open-ended generation 常用 human/LLM judge、rubric、pairwise preference。
- Perplexity 越低表示模型对下一个 token 越确定, 但必须在同一 dataset/tokenization 下比较。
- Elo 更新: $R'_A = R_A + K(O_A - E_A)$; O_A 是实际结果, E_A 是预期胜率。
- Contamination 检测: train-test overlap 报告、private/dynamic test sets、public-private 分数差、prefix completion 记忆测试。

Pitfalls

- Low perplexity 不等于 factual correctness、strong reasoning 或 user preference。
- Leaderboard score 高不等于 real deployment reliability, 尤其 safety-critical settings 不能只看 average score。
- LLM judge 不是 objective truth, 需要 human calibration 和 bias control。

- Saturation 和 contamination 会一起让公开榜单越来越不可信。

30-second exam template

LLM evaluation 难在 output open-ended、没有唯一 ground truth、capabilities multi-dimensional 且 prompt-sensitive。应先明确 use case 和 risk，再选 answer type、metric、prompt 和 deployment criteria。Perplexity 等 intrinsic metrics 适合追踪 language modelling，但不能测 factuality、reasoning 和 alignment；MMLU/GPQA/HLE 等 benchmarks 可 standardized comparison，但有 saturation 和 contamination。Open-ended tasks 可用 LM Arena 的 human pairwise preference 和 Elo 聚合，或 LLM-as-a-Judge，但要注意 judge bias 和 human calibration。

5. Long Context Modelling

Core thesis: Long-context modelling 要同时解决 positional generalisation、quadratic attention cost 和 “能放进去不等于能用好” 的 long-range reasoning 问题。

Must-know points

- Long context 指模型处理数万到百万 tokens，用于 long documents、codebases、legal materials、agent history 等。
- Three challenges: training data mismatch、quadratic attention cost、long-range reasoning difficulty。
- Transformer 需要 positional encoding，因为 self-attention 本身没有顺序信息。
- Sinusoidal PE 用不同频率的 sin/cos 给位置固定 fingerprint；低频表全局，高频表局部。
- RoPE 旋转 Q/K 向量编码位置，使相对位置通过旋转角度差进入 attention score。
- RoPE 外推问题：远位置高频旋转会 wrap around，导致相对距离噪声化。
- Position Interpolation 将 long positions compress/map 回 training length range，如 4096 映射到 2048 范围。
- YaRN 对不同 wavelength dimensions 分段 interpolation，兼顾 local details 和 global positions。
- Efficient attention 包括 sliding window、dilated window、global tokens、KV cache、KV compression。
- Evaluation 包括 Needle in the Haystack、LongBench、lost-in-the-middle。

Exam contrasts/formulas

- Input representation: $z_{pos} = x_{pos} + PE_{pos}$ 。
- Attention: $\text{Attention}(Q, K, V) = \text{softmax}(QK^\top / \sqrt{d})V$ 。
- Full attention 复杂度: $O(n^2)$ ，因为每个 token pair 都要比较。
- Sinusoidal vs RoPE: 前者把位置向量加到 token embedding；后者旋转 Q/K，让相对距离体现在 attention score。
- PI vs NTK vs YaRN: PI compresses position indices; NTK adjusts RoPE frequencies; YaRN 分段处理不同 wavelengths。
- Needle test 变量: context length 和 needle depth。

Pitfalls

- 长 context window 不等于 effective long-range reasoning。
- Needle test 只是 retrieval sanity check，不等于真实 long-document understanding。
- Sparse attention 提升效率但牺牲部分 global information integration。
- Lost-in-the-middle 说明模型常更会用 beginning/end evidence，忽略 middle evidence。

30-second exam template

Long-context modelling 让模型处理 very long input, 但核心 challenges 是 training data mismatch、full attention 是 $O(n^2)$ 、以及 long context window 不等于 effective long-range reasoning。位置上, sinusoidal PE 用 sin/cos 编码 position, RoPE 通过旋转 Q/K 让 attention score 感知 relative position; 扩展 RoPE 可用 Position Interpolation、NTK scaling、YaRN。Efficiency 上用 sparse/sliding/global attention、KV cache 或 compression。Evaluation 要看 Needle、LongBench 和 lost-in-the-middle, 判断模型是否真的能 find and use distant evidence。

6. RAG

Core thesis: RAG 把 LLM parametric memory 和外部可更新的 non-parametric memory 结合，核心流程是 retrieve evidence -> generate answer，并尽量让 output attributable。

Must-know points

- RAG = Retrieval-Augmented Generation，适合事实、最新知识、长尾实体、领域知识和 citation 需求。
- Retriever-reader architecture: retriever 从 corpus/index 找 top-k passages, reader/generator 基于 evidence answer。
- Parametric memory 是 model parameters 中的知识; non-parametric memory 是外部 document collection、database、knowledge graph。
- Sparse retrieval 如 TF-IDF/BM25，靠 term overlap，解释性强，适合精确实体。
- Dense retrieval 用 bi-encoder 把 query/passages 编成向量，用 dot product/cosine retrieval，强于 semantic matching。
- Efficient retrieval 用 MIPS、FAISS、ANN、IVF/k-means 避免逐一比较海量向量。
- RAG-Sequence 整段输出用同一组文档；RAG-Token 不同 token 可依赖不同文档。
- Attribution 指 answer claims 要能被 cited evidence 支持；有 citation 不等于 faithful。
- GraphRAG 把文档组织成实体、关系、社区/图结构，适合多跳、企业文档和全局 summarisation。
- Agentic RAG 让 agent 动态决定查询、retrieval、paraphrasing、验证和多步 tool use。

Exam contrasts/workflows

- Open-domain QA vs reading comprehension: 前者先 retrieval 再 answer；后者通常已给 passage。
- Sparse vs dense: 词面匹配、可解释、便宜 vs semantic matching、依赖训练和 vector index。
- Bi-encoder vs cross-encoder: 前者可预索引、高效初筛；后者联合编码 query-doc，precision 高但慢。
- RAG 流程: query -> retrieve top-k -> optional rerank -> generator/reader answer -> attribution/verification。
- RAG vs long context: RAG 先选相关信息、cost 低但依赖 retrieval；long context 放更多内容但可能信息过载和 lost-in-the-middle。

Pitfalls

- RAG 不能完全消除 hallucination，retrieval failure、ranking error、insufficient evidence、generator ignoring evidence 都会失败。
- 有 citation 不代表 answer 正确，citation 可能 irrelevant 或只支持部分 claims。

- Dense retrieval 不一定总比 sparse 好，精确实体和关键词场景 sparse 很强。
- GraphRAG 的代价是构图、实体关系 extraction 错误和 system complexity。

30-second exam template

RAG 通过 non-parametric memory 补充 LLM 的 parametric memory: 系统先从 external corpus retrieve top-k evidence, 再由 reader/generator generate answer, 并最好提供 attribution。Retriever 可用 sparse retrieval 如 TF-IDF/BM25, 也可用 dense bi-encoder 和 FAISS/MIPS 做 semantic retrieval; cross-encoder 常用于 reranking。RAG 能提升 factuality、freshness 和 long-tail knowledge, 但不能完全避免 hallucination。Variants 包括 RAG-Sequence/RAG-Token、GraphRAG 和 Agentic RAG。

7. Multilingual LLMs

Core thesis: Multilingual LLMs 用 shared model 服务 multilingual users, 但必须处理 data imbalance、tokenizer unfairness、transfer/interference、multilingual evaluation 和 responsible modelling。

Must-know points

- Multilingual modelling 意义: fairness、accessibility、non-English knowledge coverage、cross-lingual transfer。
- Core challenges: high-resource/low-resource imbalance、tokenization unfairness、positive transfer/negative interference、multilingual evaluation difficulty、cultural safety differences。
- FineWeb2 是面向 Common Crawl 的 multilingual high-quality web corpus pipeline。
- FineWeb2 包括 language identification、tokenisation、deduplication、quality filtering、rehydration 等步骤。
- Rehydration 利用网页在多个 crawl snapshots 中出现情况帮助判断质量。
- Data mix 决定不同语言采样比例, 影响高资源效率与低资源曝光。
- Temperature sampling 在 proportional 和 uniform 之间折中。
- Tokenization fertility 高的语言会消耗更多 token、上下文和计算预算。
- ATLAS scaling laws 强调 target-language performance 不仅取决于 target-language data, 也取决于 related languages 带来的 transfer 和 interference。
- Responsible multilingual modelling 要关注 safety transfer、local harms、data sovereignty、participatory design 和 language-community participation。

Exam contrasts/formulas

- Positive transfer vs negative interference: 一种语言帮助另一种语言 vs 一种语言训练损害另一种语言。
- Curse of multilinguality: 固定 capacity/compute budget 被更多 languages 共享, monolingual performance 可能下降。
- Proportional sampling 偏高资源; uniform sampling 平衡语言但会过采样低资源; temperature sampling 折中。
- Temperature sampling: $p_i = \frac{n_i^\alpha}{\sum_j n_j^\alpha}$; α 越小越接近 uniform, 越大越接近 proportional。
- Culturally adapted benchmark vs translated benchmark: 前者按本地文化重写; 后者简单翻译, 可能不自然、不等价、英语中心。

Pitfalls

- English performance strong 不等于 multilingual performance strong。
- Average score 高可能掩盖 low-resource languages 严重失败。
- Tokenizer 不是中立工程细节, 它决定不同语言使用模型预算的“价格”。
- “Think in English then answer target language” 不是根本解决方案,

可能丢失本地语境。

- Low-resource languages 数据少不代表语言价值低，而是 digital resource inequality 和 power imbalance。

30-second exam template

Multilingual LLM 的目标是一个 model coverage 多 languages，同时利用 shared parameters 产生 cross-lingual transfer。主要 challenges 包括 data imbalance、tokenization unfairness、positive transfer 与 negative interference、multilingual evaluation 和 cultural safety differences。FineWeb2 通过 language identification、deduplication、quality filtering 和 rehydration 构建 multilingual web data；data mix 可用 temperature sampling $p_i = \frac{n_i^\alpha}{\sum_j n_j^\alpha}$ 在 high-resource efficiency 和 low-resource exposure 间折中。ATLAS scaling laws 强调 language transfer matrix。Evaluation 时不能只看 average score，要 per-language reporting、expert validation、cultural adaptation，并考虑 data sovereignty 和 community participation。

8. Bias and Ethics

Core thesis: AI ethics 不是“模型有没有礼貌”，而是追问 LLM 在 data、training、deployment 中如何造成 harm: who benefits、who is harmed、who decides、who is accountable。

Must-know points

- LLM risk taxonomy 包括 discrimination/exclusion、information hazards、misinformation、malicious use、HCI harms、environmental/socioeconomic harms。
- Allocational harm 影响资源/机会分配，如招聘、贷款、教育、医疗。
- Representational harm 以 stereotyped、derogatory、under-representation 方式呈现 groups，强化 bias。
- Bias measurement 不能只靠一个 metric，要结合 benchmark、counterfactual testing、human evaluation。
- WEAT 用 target sets 与 attribute sets 的 cosine similarity 差测 embedding association。
- BBQ 用 ambiguous context 测 stereotype reliance，用 disambiguated context 测 bias 是否压过 correct evidence。
- Counterfactual testing 只替换姓名、性别、族裔等 demographic signals，看输出是否系统性变化。
- Mitigation 包括 preprocessing、in-training、intra-processing、post-processing。
- RLHF 可减少显性有害行为，但 reward model 会编码标注者文化视角。
- Technical mitigation 必须配合 deployment auditing、community feedback、institutional accountability。

Exam contrasts/workflows

- Allocational vs representational: 前者影响 real-world opportunities，后者影响 group portrayal。
- Measurement vs mitigation: measurement/evaluation 发现 bias，mitigation 减少 harm；metric 改善不等于 real-world harm 消失。
- WEAT 直觉: 比较词 w 与 attributes 集 X/Y 的平均 cosine similarity 差。
- Good practice: structured benchmark + domain-specific counterfactual tests + open-ended human evaluation。

Pitfalls

- 不要把 bias 简化成“training data 脏”；deployment context、annotators、institutional structures 都会制造 harm。
- 不要说 WEAT 能直接预测 real deployment bias；它主要测 embedding association。
- 不要把 RLHF 当万能 debiasing；它可能留下 subtle bias。

30-second exam template

LLM ethics 关注 system lifecycle 中的 harm 和 accountability。Bias 主要造成 allocational harm 与 representational harm，可用 WEAT、BBQ、

counterfactual testing 等 measurement methods, 但 single metric 不够。Mitigation 可在 data、training、inference/output stages 做, 但因为 bias 是 context-dependent 且会 whack-a-mole, 所以还需要 auditing、feedback 和 accountability mechanisms。

9. Safety, Watermarking and Adversarial Attacks

Core thesis: LLM safety 关注避免伤害, security 关注防止被攻击或绕过; jailbreak 常是 security failure 导致 safety failure。

Must-know points

- Safety 包括 harmful content、bias、misinformation、privacy 泄露、dangerous advice。
- Security 包括 prompt injection、jailbreak、data exfiltration、滥用、检测规避。
- Watermarking 在 generation 时嵌入 statistically detectable pattern, 用于识别 AI-generated text。
- Text watermarking 不能像 image watermarking 一样嵌不可见标记, 只能改变 token probability bias。
- Green list/red list 由上下文和 secret key 决定, 提高 green token 采样概率。
- Detection 时统计 green token ratio 是否显著高于 unwatermarked expectation。
- Watermarking limitations: short text 弱、paraphrasing/translation/summary 会破坏 signal、key leakage 可伪造/规避。
- Illicit content attack 诱导模型 generate 违法、危险或 policy-disallowed content。
- Adversarial suffix attack 优化一段 suffix, 让模型从拒绝转为服从。
- Defences 需要多层: data filtering、alignment、classifier、refusal policy、red-teaming、monitoring、watermarking。

Exam workflows/contrasts

- Watermark workflow: secret key + context 划分 green/red tokens -> 给 green tokens 加 logit bias -> generated text -> detector 做 statistical test。
- False positive: unwatermarked text 被判 watermarked; false negative: watermarked text 没检测出。
- Adversarial suffix: positive affirmation objective 让模型倾向 “Sure, here is…”; coordinate gradient search 在离散 token 上搜索 suffix。
- Closed model 更适合 watermarking control; open-source model 容易被改 decoding 或移除 watermarking。

Pitfalls

- Watermark 只能辅助来源识别, 不能阻止 harmful content。
- Training data filtering 不能删掉所有 dual-use knowledge, 否则会伤害 medicine、education、safety research 等 legitimate uses。
- Blacklist 一个 suffix 不够, 攻击者可重新优化新 suffix。

30-second exam template

Safety 是 avoid harm, security 是 prevent attacks/misuse。Watermarking 在 decoding 时用 key 把 tokens 分成 green/red list, 提高 green token

概率，detection 时看 statistical significance；但 short text、paraphrasing 和 key leakage 都会削弱它。Adversarial suffix attack 用 optimisation search 触发模型服从 malicious request，说明 safety alignment 可能依赖 fragile surface patterns，因此需要 layered defences 和 red-teaming。

10. Instruction Finetuning, RLHF and Basic RL

Core thesis: Instruction finetuning 先用 SFT 教模型按 instruction 和 format 输出, 再用 preference/verifiable rewards 优化 behaviour; coursework 里体现为 GSM8K 上从 SFT 到 GRPO。

Must-know points

- Instruction finetuning/SFT 用 prompt-completion 数据继续训练模型, 让模型学会任务格式和行为。
- SFT 不只是增加知识, 也塑造输出格式、指令跟随和 reasoning trace 风格。
- Base vs Instruct: 架构可相同, 但 Instruct 已经过 instruction alignment, 通常更会遵循 user intent。
- Basic RL: policy generates action/output, reward function 给 reward, training objective 是最大化 expected reward。
- RLHF 常见流程: 收集 preference data -> 训练 reward model -> 用 RL 优化 policy, 同时约束不要偏离 reference model。
- Coursework 使用 Qwen-2.5-0.5B/Instruct, 在 GSM8K 上做 zero-shot、SFT、GRPO。
- GRPO 不训练 neural reward model, 而用 verifiable rewards。
- Format reward 检查 output format, correctness reward 检查 numeric answer 是否正确。
- GRPO generates 多个 candidate answers, 基于 group-relative reward 优化, 不需要 critic model。
- Reward hacking 指模型学会拿 easy reward, 如 format, 而不是 solve the underlying task。

Exam contrasts/workflows

- SFT vs RL: SFT 模仿 given answer; RL 根据 reward 改变 generation policy。
- RLHF vs coursework GRPO: RLHF 通常学 reward model; GRPO 用 rule-based/verifiable rewards。
- Reference model 约束的作用: 防止 policy 为追求 reward 过度偏离原语言能力。
- 典型现象: format reward 快速上升, correctness reward 震荡/停滞, 说明模型可能优先优化 surface format。

Pitfalls

- 不要把 SFT 说成 “只提高知识”; 它也强烈塑造输出格式。
- 不要把 GRPO 等同 PPO; GRPO 用 group-relative comparison, 省掉 critic。
- reward 高不一定代表真实能力强, 可能是 reward hacking 或 Goodhart's law。

30-second exam template

Instruction finetuning 用 instruction-answer examples 训练模型, 让它学会 task format 和 CoT-style solving. RLHF 则用 human/teacher pref-

erences 训练 reward model，再用 RL 优化 policy，并用 reference constraint 防止 policy drift。Coursework 中 GRPO 用 verifiable rewards 替代 reward model，对同一 prompt generate multiple candidates 并按 group-relative reward 学习；风险是模型可能先学会 format reward，而不是真正提高 mathematical correctness。

11. Summarisation

Core thesis: Summarisation 不是简单“写短”，而是在 task type、training data、model 和 evaluation 之间权衡，把 long text compressed，同时保留 key information、coherence 和 faithfulness。

Must-know points

- Summarisation 是把一个或多个 long documents compress 成更短、coherent、informative 的 text。
- 设计 summarisation system 要考虑四件事：task type、training data、model/learning method、evaluation method。
- Single-document 总结一个文档；multi-document 总结多个相关文档。
- Multi-document 更难：冗余、冲突、一致性、长上下文。
- Generic summary 总结全文重点；query-focused summary 只总结与 query 相关内容。
- Extractive 选原文 sentences，事实更稳；abstractive summarisation generates new wording，更灵活但更易 hallucinate。
- Extractive summarisation 可建模为 sentence-level binary classification：每句是否进入 summary。
- Pointer-generator 结合 generation 和 copying，适合保留 named entities、numbers、terminology。
- Coverage mechanism 记录 past attention，减少 repeated generation。
- Summarisation evaluation 需看 relevance、fluency、coherence、faithfulness、conciseness。

Exam contrasts/formulas

- Extractive modelling: 文档 $d = (s_1, \dots, s_n)$ ，预测 $y_i \in \{0, 1\}$ 。
- Pointer-generator: $P(w) = p_{gen} P_{vocab}(w) + (1 - p_{gen}) \sum_{i:w_i=w} a_i^t$ 。
- ROUGE: 基于候选 summarisation 和参考 summarisation 的 n-gram overlap，偏 recall。
- BERTScore: 用 contextual embeddings 做 soft overlap，更能处理同义 paraphrasing。
- CNN/DailyMail vs XSum: 前者新闻 highlights，LEAD-3 很强；后者单句极限 summarisation，更抽象。
- Long-document summarisation workflow: chunk $\rightarrow$ local summaries $\rightarrow$ aggregate summaries；风险是 early omissions are unrecoverable。

Pitfalls

- 不要只答“summarisation 越短越好”；还要 faithful、informative、coherent。
- ROUGE 高不等于 factual correctness；词面相似可能掩盖语义错误。
- Extractive 不等于落后，它在医疗、法律等高风险场景很有价值。

- LEAD-3 强说明新闻数据有位置偏置，不一定说明模型真正 understanding 全文。

30-second exam template

Summarisation 是把 long document compressed into a shorter coherent summary，并保留 key information。System design 要考虑 task type、data、model 和 evaluation。任务上可分 single/multi-document、generic/query-focused、extractive/abstractive。Extractive 直接选原句，通常更 faithful；abstractive summarisation generates new wording，更 natural 但更容易 hallucinate。经典 generative model pointer-generator 用 p_{gen} 在 vocabulary generation 和 source copying 间切换，并用 coverage 减少 repetition。Evaluation 上 ROUGE 看 n-gram overlap，BERTScore 看 semantic soft matching，但都不能完全判断 faithfulness。

12. Parameter-Efficient Finetuning

Core thesis: PEFT freezes the base model, 只训练少量 task-specific parameters 来 adaptation; 三大类是 parameter composition、input composition、function composition。

Must-know points

- Full fine-tuning 对 LLM 昂贵: 要存 parameters、gradients、activations、optimizer states。
- PEFT 固定大部分 pretrained parameters θ , 只更新少量 task parameters ϕ 。
- PEFT 目标是用更低 training cost 接近 full fine-tuning 的 downstream performance。
- 三类 PEFT: parameter composition、input composition、function composition。
- Parameter composition 通过 modify/select/low-rank update 原模型 parameters 来 adapt task。
- LoRA freezes W_0 , 学习 low-rank matrices B, A , 令 $\Delta W = BA$ 。
- LoRA 常用于 attention 的 W_Q, W_K, W_V 等投影矩阵。
- Prompt tuning 学习 continuous prompt embeddings, 拼到 input 前。
- Adapters 在 Transformer 层中插入 down-projection、activation、up-projection 小 modules。
- Multi-task deployment 时可共享 base model, 只切换 LoRA/adapters/parameters。

Exam contrasts/formulas

- Parameter composition: $g_i(x) = f_{\theta_i \oplus \phi}(x)$, 例 LoRA。
- Input composition: $g_i(x) = f_{\theta_i}([\phi_i, x])$, 例 prompt tuning。
- Function composition: $g_i(x) = f_{\theta_i} \circ f_{\phi}(x)$, 例 adapters。
- LoRA: $h = [W_0 + \Delta W]x = [W_0 + BA]x$ 。
- Adapter: $f_{\phi}(x) = W_U \sigma(W_D x)$, bottleneck 降低 parameter count。
- LoRA rank r 控制 capacity 和 parameter count; r 越大越强但越贵。

Pitfalls

- PEFT 不是不训练; ICL 才是不更新 parameters。
- Prompt tuning parameters 少, 但会占 context 并增加 inference cost。
- LoRA can be merged into weights, inference efficiency 通常好; adapter 增加额外 module calls。
- Rank 太低或应用层选错会限制 LoRA 表达能力。

30-second exam template

PEFT freezes pretrained model, 只训练少量 task parameters, 解决 full fine-tuning 在 LLM 上 GPU memory 和 storage 过高的问题。三类方法

是 parameter composition、input composition、function composition, 分别对应 LoRA、prompt tuning、adapters。LoRA 用 $\Delta W = BA$ low-rank update 原 weights, parameters 少且 mergeable; prompt tuning 把 adaptation information 放进 input; adapters 插入 bottleneck modules, 稳定但 inference 多一步。

13. Distillation

Core thesis: Knowledge distillation 用 strong teacher 生成 data、labels、feedback 或 distributions, 再训练 student 学到能力, 常用于把 proprietary strong model 能力转给 small/open-source model。

Must-know points

- Teacher 通常更强、更大或 proprietary; student 更小、更便宜、更可控。
- KD 不只是 compression, 也用于 capability transfer、instruction following、domain adaptation、self-improvement。
- Proprietary LLM 强但 black-box、expensive、有 privacy risk; open-source 可控但通常 weak。
- KD recipe: 选 skill/domain/template -> 准备 seed knowledge -> generate distillation knowledge -> 训练 student。
- 六种获取知识: labelling、expansion、data curation、feature、feedback、self-knowledge。
- Labelling: 给定 x , teacher generates y 。
- Expansion: teacher generates new input x 和对应 output y 。
- Feature: 用 teacher internal representations/分布, 通常需要 white-box。
- Feedback: teacher evaluates student output, 可 generate preference data。
- 转移方法包括 SFT、divergence/similarity、RL、ranking optimisation/DPO。

Exam contrasts/formulas

- KD 数据: $D_I^{kd} = \{\text{Parse}(o, s) \mid o \sim p_T(o \mid I \oplus s), s \in S\}$ 。
- Student 目标: $L = \sum_I L_I(D_I^{kd}; \theta_S)$ 。
- Black-box KD: 只用输出/evaluation; white-box KD: 用 hidden states、logits、分布。
- SFT vs divergence: SFT 学 discrete answer; divergence 拟合 teacher 完整 output distribution。
- RL vs DPO: RL 先学 reward model 再优化 policy; DPO 直接用 preference pairs 优化。

Pitfalls

- KD 不一定需要 teacher parameters; 很多 LLM KD 只靠 API 输出。
- Teacher-generated data 不一定正确, student 会继承 hallucination、bias、low diversity。
- White-box 方法信息更丰富, 但 proprietary strong models 通常拿不到 internal representations。
- Student capacity、training data 和 objective function 会限制 distillation upper bound。

30-second exam template

Knowledge distillation 是用 teacher 产生 trainable knowledge, 再让 stu-

dent 学习。流程是确定 target skill 和 template, 准备 seed examples, 让 teacher generate distillation data, 最后用 SFT、distribution alignment、RL 或 DPO 训练 student。Black-box KD 适合 API teacher, white-box KD 可利用 logits/hidden features; 主要风险是 teacher errors 和 bias 被 student 继承。

14. Reasoning During Decoding

Core thesis: Decoding-time reasoning 不改 model parameters, 而是在 inference/decoding 时用 CoT、sampling/voting、search、reflection 或 tools 提高 answer quality。

Must-know points

- Reasoning 指用事实/假设进行结构化思考, 得出结论或解释。
- 本讲重点是 inference-time/decoding-time reasoning。
- Chain-of-thought 让模型 generate intermediate reasoning steps 再 answer。
- CoT 局限是 linear path 走错后可能一路错下去。
- Self-consistency 采样多条 reasoning chains, 对 final answer 投票。
- Temperature 控制采样多样性; 太高会增加噪声。
- Tree of Thoughts 显式探索 branching reasoning paths。
- ToT 组件: thought decomposition、generator、state evaluator、search algorithm。
- Self-reflection: generate answer $\rightarrow$ feedback $\rightarrow$ revise, 可迭代多轮。
- External verifier/solver 调用 theorem prover、code executor、symbolic solver 等验证或求解。

Exam contrasts/workflows

- Self-consistency: 采样 $r_1, \dots, r_n$ 得到 $a_1, \dots, a_n$, 选 majority answer。
- CoT vs self-consistency vs ToT: single linear reasoning path vs multiple complete chains with voting vs tree search over intermediate states。
- ToT search: 定义 thought granularity $\rightarrow$ generate candidates $\rightarrow$ evaluate frontier states $\rightarrow$ BFS/DFS search。
- Self-reflection 需要 actionable、specific feedback; 笼统反馈无用。
- External verifier 更可靠, 但难点是 natural language 到 formal representation。

Pitfalls

- Majority vote 不是 proof of correctness; 多条 chains 可能共享同一 bias。
- ToT 很贵, 而且 problem-specific。
- Self-reflection 不一定提升, 模型可能无法发现自己的错。
- Tools 不会让模型自动变聪明, 模型仍要 correctly call tools and interpret tool feedback。

30-second exam template

Inference-time reasoning 通过 decoding strategies 而不是 training 来提升 reasoning。CoT generates one reasoning chain; self-consistency samples multiple CoT chains 并对 answer 投票; ToT 把 reasoning 变成

search, 包含 thought generator、evaluator 和 BFS/DFS; self-reflection 用 specific feedback 迭代修改; external verifier 用 external tools 检查 correctness, 但需要正确 formalise problem。

15. Mechanistic Interpretability

Core thesis: Mechanistic interpretability 试图 reverse-engineer 模型内部机制, 解释 features、neurons、layers、circuits 如何 causally produce behaviour。

Must-know points

- Behavioral evaluation 看 input-output behaviour; mechanistic interpretability 看 internal computation。
- 方法类型包括 behavioral、attributional、concept-based、mechanistic。
- Feature 是 activation space 中表示某 concept/pattern 的 direction。
- Concept 是人类语义概念, 可能由多个 features 实现。
- Linear probe 检测信息是否可线性读出, 但不能证明模型使用该信息。
- Monosemantic neuron 主要对应一个清晰 feature。
- Polysemantic neuron 对多个不相关 features 激活。
- Superposition 指低维 space 用 overlapping directions 编码多于 dimensions 数的 sparse features。
- SAE 从 activations 中学习 sparse feature dictionary, 帮助解开 superposition。
- Circuit 是多个 features/neurons/heads/layers 共同实现某个计算。

Exam contrasts/formulas

- Dot product 衡量 vector direction alignment, 也可看作 feature projection。
- SAE: $c = \text{ReLU}(Mx + b)$ 编码 sparse feature activations; $\hat{x} = M^T c$ reconstruction activation。
- SAE loss = reconstruction loss + L1 sparsity penalty。
- Probe vs SAE: probe 测信息可读出; SAE 学 sparse feature basis。
- Superposition vs polysemanticity: 前者是 representational compression mechanism, 后者是 neuron-level ambiguity phenomenon。
- Universality hypothesis: 不同模型可能学到相似 features/circuits, 但必须实证验证。

Pitfalls

- Neuron 不等于 feature; feature 可分布在多个 neuron 上。
- Probe 结果是 correlation, 不是 causal evidence。
- SAE 学到的 feature 不一定真实可解释, 需要验证。
- Strong universality 不能默认成立, 架构、数据、规模都会影响内部机制。

30-second exam template

Mechanistic interpretability 目标是解释模型内部如何产生 behaviour, 而不只看 input-output。核心概念包括 feature、monosemanticity/polysemanticity、superposition、SAE 和 circuits。Superposition

说明模型用 overlapping directions compress 许多 sparse features, 导致 neurons polysemantic; SAE 用 sparse coding 学更可解释的 feature dictionary; 最终要用 causal intervention 证明某 circuit 真参与 computation。

16. Syntactic Parsing

Core thesis: Syntactic parsing 把 sentence 变成 explicit structure, 用 constituency 或 dependency 表示 words 如何组成 phrases 或 dependency relations, 并用 grammar、PCFG 或 neural chart parsing 找到 high-scoring valid tree。

Must-know points

- Syntax 描述 words 如何组成 phrases 和 sentences, 帮助处理 structure、ambiguity 和 long-distance relations。
- Parsing 是为句子构建 syntactic structure, 如 constituency tree 或 dependency tree。
- Constituency parsing 关注 phrase boundaries 和 hierarchy, 如 NP、VP、S。
- Dependency parsing 关注词与词之间 head-dependent 关系。
- Grammar 要 cover language phenomena, 同时避免 overgeneration 和 undergeneration。
- CFG 包含 non-terminals、terminals、规则和 start symbol。
- PCFG 给 CFG 规则加概率, 同一左侧 rule probabilities 和为 1。
- 一棵 PCFG parse tree 的概率是所有使用 rule probabilities 的乘积。
- Supervised parsing 依赖 treebank; unsupervised grammar induction 没有 gold trees, 更难。
- Transformer parser 通常先编码 sentence, 再给 spans 或 actions 打分, 并用 global decoding 保证 valid parse tree。

Exam contrasts/formulas

- Constituency vs Dependency: 前者看 phrase span 和 hierarchy; 后者看 head-dependent edges。
- PCFG tree probability: $P(\text{tree}) = \prod_{\text{rules}} P(\text{rule})$ 。
- Parsing evaluation: constituent precision、recall、F1; dependency 常用 UAS/LAS。
- Labelled vs unlabelled constituent evaluation: 前者要求 span 和 label 都对; 后者只看 span。
- CYK/chart parsing: 对每个 span 尝试 split, 组合左右子 span 得分, 用 backpointers 恢复最佳树。
- Span-based neural parsing: Transformer 编码 $\rightarrow$ 所有 $O(n^2)$ spans 打分 $\rightarrow$ chart decoding 找最高分 valid parse tree。

Pitfalls

- Transformer 不会天然输出合法 parse tree, 通常还需要 chart parsing 或约束解码。
- 局部 span 高分不代表能组成全局 valid parse tree, 因为 spans 可能交叉或冲突。
- PCFG 简单可解释, 但独立性假设强, 难捕捉上下文和长距离依赖。
- Parsing 不只是判断句子是否合法, 而是找最可能或最有用的结构。

30-second exam template

Syntactic parsing 的目标是给 sentence 建立 structure, 帮助模型处理 phrase hierarchy、dependency relations 和 ambiguity。Constituency parsing 把 sentence 分成 nested constituents, 如 NP、VP; dependency parsing 则表示 words 之间的 head-dependent edges。传统方法可用 CFG/PCFG, 其中 PCFG 给 grammar rules 加 probabilities, 整棵 tree probability 是 rule probabilities 乘积。现代 Transformer parser 用 contextual representations 给所有 spans 或 actions 打分, 但 local high-scoring structures 不一定 global valid, 所以需要 chart/CYK-style dynamic programming 和 backpointers 找 highest-scoring valid parse tree。Evaluation 中 constituency parsing 常用 precision、recall、F1; dependency parsing 常用 UAS 和 LAS。

17. Perceptually-Grounded NLP

Core thesis: Perceptually-grounded NLP 的核心是: language meaning 不只来自 text co-occurrence, 还要连接到 vision、action、environment 和 social interaction。

Must-know points

- Multimodal AI 整合 text、images、audio、video 等 data 来提升 understanding、prediction 和 decision-making。
- 本课程重点关注 language 和 vision 的结合, 即 Vision+Language Models。
- Text-only representation 只能从 text co-occurrence 学习 meaning, 缺少 direct perception of world、object attributes 和 action consequences。
- Symbol grounding problem 问的是: 如果 symbols 只由其他 symbols 解释, meaning 如何最终连接到 real-world referents。
- Chinese Room argument 说明仅按 rules 操作 symbols 不等于真正 understand meaning。
- Chinese/Chinese dictionary-go-round 说明只在 symbols 内部循环会无法 ground in real experience。
- Communication grounding 指对话双方共享 mutual knowledge、beliefs、assumptions。
- Perceptual grounding 要让 language system 通过 perception 和 world interaction 获得 referents。
- Experience Grounds Language/World Scopes 强调 learner 能访问的 data 和 environment scope 决定可学能力。
- Reporting bias 指 text 倾向记录 worth-mentioning/unusual information, 省略显而易见的 visual facts。

Exam contrasts/workflows

- Amodal symbols vs grounded symbols: 前者只在 symbol system 内操作; 后者与 perception、categorisation 和 real-world referents 相连。
- Visual grounding 的价值: 提供 objects、attributes、spatial relations 和 actions 信号, 补充 text-only learning。
- World scope hierarchy 启发: 增加 modalities、action、互动会改变模型可学习的信息和归纳偏置。
- VLM 任务例子: image captioning、VQA、referring expression、visual dialogue、visual storytelling。

Pitfalls

- 不要说 “multiple modalities 只是多一个 input format”; 重点是 meaning、perception 和 world experience 的连接。
- 扩大 LLM 数据和规模能提升文本能力, 但不完全解决 grounding。
- Grounding 有多个含义: communication grounding 和 symbol/perceptual grounding 要分清。

30-second exam template

Perceptually-grounded NLP 的动机是 text-only models 只从 symbol co-occurrence 学习, 受到 symbol grounding problem 和 reporting bias 限制。Language meaning 与 perception、action、environment 和 social interaction 相关; 因此 VLM 用 visual signals 把 language representations 连接到 objects、attributes、spatial relations 和 events。World scopes 进一步说明, learner 能访问的 experience scope 越丰富, 可学到的 language ability 和 world knowledge 也不同。

18. Vision-Language Models

Core thesis: VLM 用 pretrained vision encoder、LLM backbone 和 fusion/projector 连接 images 与 language; 核心考点是 architecture choices、connector necessity、training data、SmolVLM 和 evaluation。

Must-know points

- VLM 整合 vision 和 language data, 使模型能基于 images 和 text 完成 understanding、generation 和 decision-making。
- 常见架构包括 cross-encoding、cross-attention、visual prefix。
- Cross-encoding 把 vision 和 language tokens 一起 input 同一 Transformer。
- Cross-attention 让 language model 通过 attention 访问 vision representations。
- Visual prefix 把 vision representations 投影成 LLM 可处理的 prefix tokens, 让 text LLM 在 visual context 条件下 generate。
- Visual prefix 组件: pretrained vision encoder、projector、text-only LLM backbone、image-text training data。
- Vision encoder 常用 ViT 或 CLIP/SigLIP 等 vision-language contrastive encoders。
- ViT 把 images 切成 patches; patch 数越多, vision tokens、context length 和 GPU memory cost 越高。
- VLM 通常在 language tokens 上用 next-token cross-entropy, 不对 visual tokens 算 next-token loss。
- VLM 数据包括 noisy web image-text data、captioning、OCR、PDF/charts/tables、multi-image/video、instruction data。

Projector/Connector 必背

- Projector 必要性: vision encoder 输出 dimensionality 和 semantic space 不同于 LLM hidden states, 需要映射到 LLM 可处理空间。
- Connector 还可能 compress visual tokens, 控制 context length 和 memory cost。
- 选择包括 linear projection、MLP、average pooling、CNN/convolutional mapping、attention pooling/cross-attention with learnable queries。
- Simple connector 便宜稳定但表达力弱; complex connector 更强但 cost 高、训练更不稳定。
- Compression 提高效率但可能丢失细粒度 vision 细节, 尤其损害 OCR。
- 训练常先 freeze LLM/vision encoder, 只训练 projector; 后续再 partial/full finetune, 减少 catastrophic forgetting。

SmolVLM 必背

- SmolVLM 使用 SigLIP vision encoder、SmolLM2 language backbone 和 projector。
- Patch 数量公式: $NP = (H/P)^2$, 例如 224x224 images、16x16 patch 有 196 patches。

- Pixel shuffle compresses visual token count, 降低 spatial resolution、提高 representation density, 在 efficiency 和 spatial detail 间折中。
- SmolVLM 三阶段: adaptation stage 学 projector; image stage 强化 image/OCR/document understanding; video stage 扩展到 multi-image 和 video。
- Adaptation stage 通常 freeze language model, 使用 noisy web data 和 captioning 数据。
- Image stage 可 finetune 整个模型, 并混入文本数据以保留 instruction-following。
- Edge VLM 重要, 因为低 latency、privacy、本地 deployment 和 cost 控制都要求小型高效模型。

Evaluation

- Extrinsic evaluation 用具体 task/application dataset 评 VLM performance。
- Intrinsic evaluation 直接评 representation quality, 如 attention pattern、brain alignment、representational quality。
- Task-specific evaluation 明确但 transferability 有限; multi-task evaluation coverage 广但 cost 高、aggregation 难。
- 例子: FOIL captions、SVO-Probes、Winoground、VLMEvalKit、MMMU。
- Winoground 测 visio-linguistic compositionality, 要求 understanding word order、relations 和 compositional meaning。

Pitfalls

- 不要把 projector 只说成 “dimensionality mapping”; 它还承担 semantic alignment、token compression、training stability 影响。
- Compressing visual tokens 不总是好, OCR 和 fine-grained localisation 需要保留 local information。
- VLM 不等于完整 real-world multimodal agent; 多数 VLM 缺少 action、environment 反馈、声音、触觉和长期交互。

30-second exam template

Visual prefix VLM 用 vision encoder 产生 visual tokens, 经 projector 映射到 LLM hidden space, 作为 prefix 条件化 text LLM, 并在 language tokens 上用 cross-entropy 训练。Projector 必要是因为 vision features 和 LLM token space 在 dimensionality 与 semantics 上都不同; 它可用 linear/MLP/pooling/CNN/attention 实现, 在 efficiency、expressivity、stability 和 information preservation 之间权衡。SmolVLM 体现了这一 pipeline: SigLIP + SmolLM2 + projector, 使用 pixel shuffle 控制 token count, 并通过 adaptation、image、video 三阶段训练扩展能力。Evaluation 要区分 extrinsic task benchmarks 和 intrinsic representation evaluation。

19. Text-Only AI Agents

Core thesis: AI agent 是通过 sensors perceive environment、通过 actuators act on environment 的系统；LLM agent 则把 goal、state、plan 和 actions 主要表示为 text 与 tool use。

Must-know points

- Russell & Norvig 定义: agent 通过 sensors perceive environment, 通过 actuators act on environment。
- 现代 LLM agent 仍是 perception、decision-making、action, 只是 environment 常是 text、tools、API 或 software system。
- LLM-based agent 组成: LLM、textual input/output representation、wrapper/framework、tools、memory、workflow control。
- Agentic problem-solving process: Get the Mission、Scan the Scene、Think it Through、Take Action、Learn & Get Better。
- Framework != LLM: 用户通常与包装 LLM 的 agent framework 交互, 而不是裸 LLM。
- Modality gap: 非文本 data 必须转成 LLM 可用 representation, text-only LLM 无法直接处理 raw images、audio 或 GUI。
- Context length limit 会导致 long task history、tool results、documents 被 truncated 或 cost 上升。
- Lack of memory 指 LLM 本身没有真正 long-term memory, long-term state 需要 RAG、database 或 external memory 模拟。
- Action space 是 agent 可选择和执行的 actions 集合, 决定 capability、safety 和 recovery options。
- Agent reliability 难, 因为 planning、tool use、context management 和 environment interaction 任何环节都可能出错并 cascade。

Agentic patterns

- Single-call execution: 一次 prompt 解决简单任务; 简单但不适合复杂多步。
- Prompt chaining: 把任务拆成顺序 prompts; modules 化、易调试, 但会 error propagation。
- Routing: 根据 input 类型/意图/safety 约束选择流程; 动态但 router 会成为 failure point。
- Parallelisation: 独立子任务并行执行, 再 join/merge; 快但需要整合、deduplication、处理冲突。
- Tool use: LLM 决定何时调用外部函数、数据库或 API, 并处理 observation/result。
- MCP 是 Model Context Protocol, 用 client-server model 标准化 LLM 与 tools/resources/prompts 的通信。

Exam contrasts/workflows

- Tool use workflow: 定义 tool -> LLM generates structured call -> execute tool -> observation 回到 model -> final answer 或 continue。
- Tool definition 包含名称、功能描述、parameters schema、调用约束。

- Routing 可用 rule-based、classifier-based、embedding-based、LLM-based。
- Tool use vs RAG: RAG 是 retrieval external information 的特定模式; tool use 更广, 可调用 database、API、code、search 或 RAG。

Pitfalls

- 很多 “agent” 只是 LLM wrapper, 不要被 hype 混淆; 要看是否有 state、actions、feedback 和 environment interaction。
- Chaining 提高结构化, 但早期错误会传给后续步骤。
- Tool use 扩展 capability, 也带来 structured-output errors、tool execution failures 和 permission/safety risks。
- Agent 能执行 real actions 时必须控制 action space、permissions 和 parameter validation。

30-second exam template

AI agent 可定义为通过 sensors perceive environment、通过 actuators act on environment 的系统。LLM agent 用 language 表示 goal、state、plan 和 actions, 通常由 LLM、wrapper、memory、tools 和 workflow control 组成。常见 reliability patterns 包括 single-call、prompt chaining、routing、parallelisation 和 tool use: single-call 简单但弱, chaining 可拆解但会 propagate errors, routing 动态但可能 misclassify, parallelisation 快但要 merge results, tool use 扩展 capability 但增加 execution risk。MCP 是标准化 agent 与 external tools/resources 通信的 protocol。

20. Vision-Based Agents and OpenCUA

Core thesis: Vision-based agents 把 agent 从 text-only 扩展到 images、screen 和 embodied environment; Computer Use Agent 的核心是用 visual state、trajectory data 和 structured actions 完成 long-horizon computer-use tasks。

Must-know points

- VLM-based agent input 可包含 vision+text, output 通常仍可用 text/code/action 表示。
- Real world 包含 images、video、charts、tables 和 GUI, 仅靠 text 会丢失大量 state information。
- Visual input 可避免把 GUI/DOM/image details 全部转成超 long-document, 从而缓解 context saturation。
- Embodied AI 是能通过 vision 或其他 perception streams 与 simulated/real environment 交互的 agents。
- Computer Use Agent 能控制电脑和应用程序, 根据语言指令规划并执行点击、input、滚动等操作。
- Vision-based CUA 的 state 通常是当前 screen screenshot, 而不是 DOM/HTML。
- Trajectory representation: $I, s_0, a_0, s_1, a_1, \dots, s_T, a_T$ 。
- I 是 language instruction, s_t 是 state/screenshot, a_t 是 action, 如 click(x,y)、type(text)、scroll、press key、terminate。
- Termination action 重要, 因为 agent 需要知道何时停止, 否则会循环或执行多余 actions。
- Action representation 要 structured, 才能被系统 reliable execution 和 evaluation。

Training data and OpenCUA

- Vision-based agents 需要 trajectories, 而不是单纯 image-caption data。
- AgentNet 收集人类电脑操作数据, 包括 screen recording、鼠标键盘 input、accessibility trees。
- AgentNet 跨 Ubuntu、Windows、macOS, coverage 多 applications 和 websites, 提高 cross-environment generalisation。
- 原始 screen recordings 不能直接训练, 因为 frame rate 高、actions dense 且 low-level noise 多。
- Data curation 包括 action reduction 和 state-action matching。
- Action reduction 降低 action density, 只保留 high-level useful signal。
- State-action matching 从 video 中 extract 能预测当前 action 的 key frames, 与 actions 配对。
- OpenCUA 发现直接用 (I, s_t, a_t) finetune 效果不好, 因为模型难以 grounding 和 reasoning over trajectory。
- Long-form CoT for CUA 提供 observation、thought、action 中间 supervision。
- OpenCUA 三层: L3 observation + thought + action; L2 thought

+ action; L1 action only。

- OpenCUA 还加入 action history 和 visual history，帮助模型追踪 task progress。
- 训练阶段：Stage 1 GUI screen visual grounding; Stage 2 OS environment planning + reasoning。

Evaluation

- CUA evaluation 核心看 agent 是否能根据 language instruction 在 real/simulated computer environment 中达到 final goal。
- Success rate 衡量任务成功完成比例，但不解释中间失败。
- 还要 evaluate visual grounding、loops、backtracking、intermediate goals、failure causes、safety。
- OSWorld evaluates multimodal agents on open-ended tasks in real computer environments。
- OSWorld-Verified 修复网站变化、指令歧义、evaluation 函数脆弱等问题，使评测更 reproducible。
- WindowsAgentArena、ScienceBoard、Online-Mind2Web、ComputerUseArena 分别测试不同 OS、科学工作流、live websites、human-centric open-ended tasks。
- Future challenges: visual grounding gap、long-horizon modelling、credit assignment、security & reliability。

Pitfalls

- CUA 不是普通 search agent; 它要连续 observe interface、execute actions、处理 state changes 和 error recovery。
- 不使用 DOM/HTML 会更接近真实 visual operation，但也更难，因为要从 screenshot 中 grounding。
- Success rate 不足以 evaluate long tasks，需要分析 failure steps 和 safety risks。
- Synthetic CoT 有用，但也可能引入模型教师的偏差和不真实 reasoning。

30-second exam template

Computer Use Agent 是能根据 language instruction observe screen、plan 并 execute computer actions 的 vision agent。State 通常是 screenshot s_t ，actions 是 click、type、scroll、terminate 等 structured operations，完整任务表示为 trajectory $I, s_0, a_0, \dots, s_T, a_T$ 。OpenCUA 先用 AgentNet 收集和清理 human computer-use trajectories，再用 long-form CoT 构造 observation/thought/action supervision，并训练 VLM 做 GUI grounding 与 OS planning/reasoning。CUA evaluation 困难，因为 tasks long-horizon、environment changes 大、success rate 不足以解释 failure，还要看 grounding、loops、backtracking、credit assignment 和 safety reliability。

Final rapid recap

- MT: $T^* = \arg \max_T p(T \mid S)$, parallel corpora + encoder-decoder + adequacy/fluency/ChrF/COMET。
- QA: RC 给 passage; ODQA 先 retrieve 后 read/generate; FiD 在 decoder 融合多 passage。
- Data: 来源、筛选、deduplication、混合、synthetic 和伦理共同决定 LLM upper bound。
- Evaluation: 没有 one true metric; use case、ground truth、metric、prompt、deployment criteria 一起定。
- Long context: 位置 generalisation + $O(n^2)$ attention + lost-in-the-middle。
- RAG: parametric memory + non-parametric memory; retrieval quality 和 attribution 一样重要。
- Multilingual: data mix、tokenizer fertility、transfer/interference、cultural adaptation in evaluation。
- Ethics/Safety: harm taxonomy、bias measurement/mitigation、watermark、jailbreak、adversarial suffix。
- SFT/RLHF: SFT 学格式和行为; RLHF/GRPO 用 reward 优化 policy, 但有 reward hacking。
- Summarisation: compression + coverage + 连贯 + faithfulness; ROUGE/BERTScore 不能完全测事实性。
- PEFT: LoRA 改 weights, prompt tuning 改 input, adapters 加函数。
- Distillation: teacher generation 知识, student 用 SFT/divergence/RL/DP 学。
- Reasoning: CoT、自一致性、ToT、reflection、external verifier 都是 inference-time 策略。
- MI: feature、superposition、SAE、circuits; probe 不是 causal 证明。
- Parsing: constituency/dependency; PCFG; span scoring + chart/CYK decoding。
- Grounding/VLM: symbol grounding + visual prefix + projector + SmolVLM data/stages + VLM evaluation。
- Agents: agent = sense/decide/act; text-only patterns 到 vision-based CUA, 关键是 trajectories 和 robust evaluation。